

BÁO CÁO CODE TEST

Golden Owl DevOps Internship Challenge

Thực hiện: Nguyễn Minh Huy

Ngày hoàn thành: 30/05/2026

Repository: github.com/minhhuy27/goldenowl-devops-internship-challenge

Live Application: goldenowl-devops-alb-903237556.ap-southeast-1.elb.amazonaws.com

MỤC LỤC

1. Tổng quan.....	1
2. Containerization (Docker).....	3
3. CI pipeline.....	4
4. CD pipeline.....	7
5. Infrastructure as Code (Terraform).....	9
6. Load Balancer & Auto Scaling.....	10
7. Ứng dụng:.....	12
8. Kết luận.....	13

1. Tổng quan

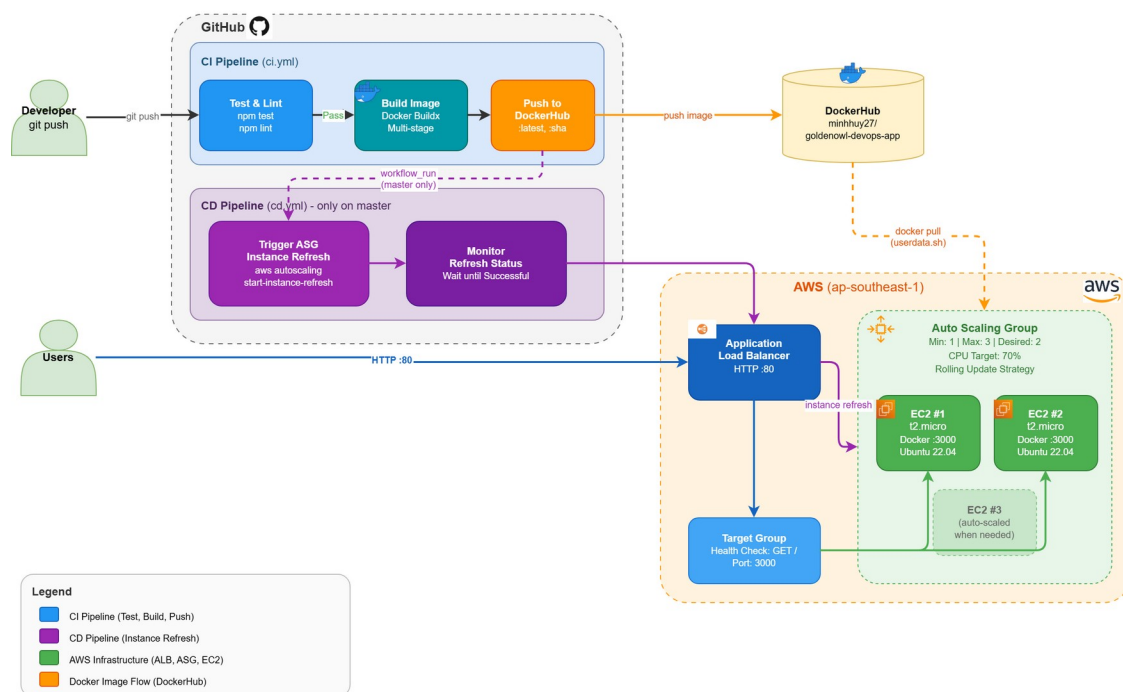
1.1 Mục tiêu

Xây dựng hệ thống CI/CD hoàn chỉnh và hạ tầng cloud trên AWS cho ứng dụng Node.js, đáp ứng các yêu cầu:

Yêu cầu	Trạng thái
Fork repository về GitHub cá nhân	Hoàn thành
Dockerize ứng dụng Node.js	Hoàn thành
CI/CD pipeline với GitHub Actions + DockerHub	Hoàn thành
Tự động chạy CI test khi push feature branch	Hoàn thành
CD deploy lên AWS	Hoàn thành
Visual flow diagram	Hoàn thành (draw.io)
Load Balancer	Hoàn thành (AWS ALB)
Auto Scaling	Hoàn thành (AWS ASG)

1.2 Kiến trúc tổng quan

Golden Owl DevOps - CI/CD Pipeline & AWS Architecture



1.3 Tech stack

Thành phần	Công nghệ
Application	Node.js 18 + Express
Container	Docker (multi-stage build, node:18-alpine)
CI Pipeline	GitHub Actions (ci.yml)
CD Pipeline	GitHub Actions (cd.yml) + AWS CLI
Container Registry	DockerHub
Infrastructure	Terraform
Load Balancer	AWS Application Load Balancer (ALB)
Auto Scaling	AWS Auto Scaling Group (ASG)
Compute	AWS EC2 (t2.micro)

2. Containerization (Docker)

2.1 Chiến lược Multi-stage build

Ứng dụng sử dụng **Dockerfile multi-stage** để tối ưu kích thước image và bảo mật:

```
Dockerfile
# Stage 1: Build & Test
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm test

# Stage 2: Production
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force
COPY . .
EXPOSE 3000
USER node
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/ || exit 1
CMD ["node", "index.js"]
```

2.2 Điểm nổi bật

Tính năng	Chi tiết	Lợi ích
Alpine-based	Sử dụng node:18-alpine	Giảm kích thước của image
Non-root	USER node	Ngăn attacker escalate privileges
Health check	Kiểm tra GET / mỗi 30s	Container tự restart nếu unhealthy
Test in build	RUN npm test ở stage 1	Image chỉ được tạo khi test pass
Deterministic	npm ci thay npm install	Cài đúng phiên bản từ lockfile
Layer caching	COPY package*.json trước	Chỉ cài lại khi dependencies thay đổi

2.3 Minh chứng

minhhuy27/goldenowl-devops-app

Last pushed about 13 hours ago • Repository size: 88.4 MB • ☆0 • ↓ 300

Add a description

Add a category

Docker commands

To push a new tag to this repository:

```
docker push minhhuy27/goldenowl-devops-app:tagname
```

Tag	OS	Type	Pulled	Pushed
247fc37		Image	less than 1 day	about 13 hours
latest		Image	less than 1 day	about 13 hours
master		Image	less than 1 day	about 13 hours

3. CI pipeline

3.1 Workflow

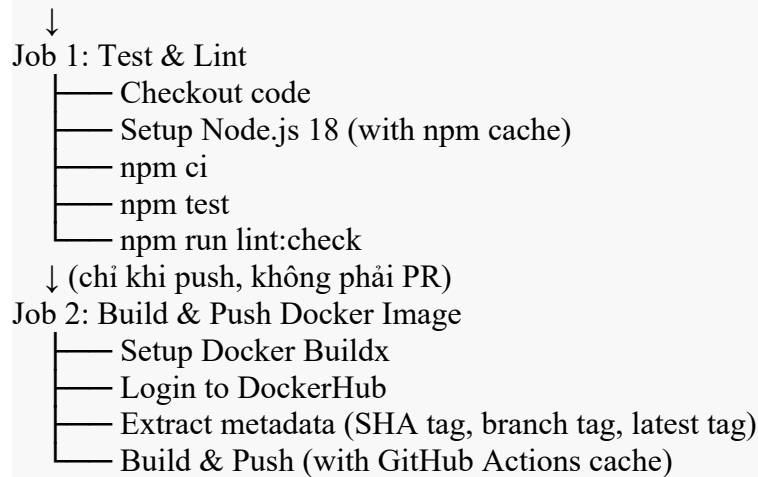
File: .github/workflows/ci.yml

Trigger:

```
on:
  push:
    branches:
      - 'feature/*'
      - master
    paths-ignore:
      - '*.md'
      - 'docs/**'
      - 'terraform/**'
      - '.gitignore'
      - 'LICENSE'
  pull_request:
    branches:
      - master
    paths-ignore:
      - '*.md'
      - 'docs/**'
      - 'terraform/**'
      - '.gitignore'
      - 'LICENSE'
```

3.2 Luồng CI

Developer push → GitHub Actions trigger



3.3 Chiến lược Tag

Tag	Điều kiện	Ví dụ
SHA	Mọi push	a1b2c3d
branch	Mọi push	master, feature-ci-pipeline
latest	Chỉ khi push vào master	latest

Việc tag bằng SHA giúp **traceability** - từ image có thể trace ngược về commit cụ thể.

3.4 Tối ưu

- **npm cache:** cache: 'npm' + cache-dependency-path → skip download nếu lockfile không đổi.
- **Docker layer cache:** cache-from: type=gha + cache-to: type=gha,mode=max → tái sử dụng Docker layers từ GitHub Actions cache.

3.5 Minh chứng

❖ CI Pipeline chạy thành công trên feature branch

The screenshot shows the GitHub Actions interface for a repository named "goldenowl-devops-internship-challenge". The selected workflow is "Add CI pipeline with GitHub Actions #1". The run was triggered 20 hours ago by user "minhhuy27" on the "feature/ci-pipeline" branch. The status is "Success" with a total duration of 1m 6s. The workflow consists of two jobs: "Test & Lint" (16s) and "Build & Push Docker Im..." (43s). The "Annotations" section shows 2 warnings related to deprecated Node.js 20 actions.

CI Pipeline

✓ Add CI pipeline with GitHub Actions #1

Re-run all jobs Latest #3

Summary

All jobs

- ✓ Test & Lint
- ✓ Build & Push Docker Image

Run details

- Usage
- Workflow file

Re-run triggered 20 hours ago

Job	Status	Total duration	Artifacts
minhhuy27 -> cb1a704 feature/ci-pipeline	Success	1m 6s	-

ci.yml on: push

Test & Lint 16s Build & Push Docker Im... 43s

Annotations

2 warnings

Test & Lint
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-node@v4. Acti...
[Show more](#)

❖ CI Pipeline chạy thành công trên master branch

The screenshot shows the GitHub Actions interface for the same repository. The selected workflow is "Merge pull request #2 from minhhuy27/feature/cd-pipeline #6". The run was triggered via push 19 hours ago by user "minhhuy27" on the "master" branch. The status is "Success" with a total duration of 59s. The workflow consists of two jobs: "Test & Lint" (18s) and "Build & Push Docker Im..." (35s). The "Annotations" section shows 2 warnings related to deprecated Node.js 20 actions.

CI Pipeline

✓ Merge pull request #2 from minhhuy27/feature/cd-pipeline #6

Re-run all jobs

Summary

All jobs

- ✓ Test & Lint
- ✓ Build & Push Docker Image

Run details

- Usage
- Workflow file

Triggered via push 19 hours ago

Job	Status	Total duration	Artifacts
minhhuy27 pushed -> e7be634 master	Success	59s	-

ci.yml on: push

Test & Lint 18s Build & Push Docker Im... 35s

Annotations

2 warnings

Test & Lint
Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/checkout@v4, actions/setup-node@v4. Acti...
[Show more](#)

4. CD pipeline

4.1 Workflow

File: .github/workflows/cd.yml

Trigger:

```
on:
  workflow_run:
    workflows: ["CI Pipeline"]
    types:
      - completed
    branches:
      - master
```

CD chỉ chạy khi CI pipeline chạy **thành công** (conclusion == 'success') - Trên branch **master**.

4.2 Luồng CD

CI Pipeline chạy thành công trên master

↓ (workflow_run trigger)

Deploy to ASG

- ├── Configure AWS Credentials (access key + secret key)
- ├── Trigger ASG Instance Refresh
 - └── aws autoscaling start-instance-refresh
- ├── Wait for Instance Refresh to complete
 - └── Polling loop: describe-instance-refreshes mỗi 30s

4.3 Quy trình Rolling Update

Khi Instance Refresh được trigger:

1. ASG launch EC2 instance mới
2. EC2 mới chạy *userdata.sh*:
 - ├── Cài đặt Docker
 - ├── docker pull <image>:latest
 - └── docker run -p 3000:3000 <image>:latest

3. ALB health check kiểm tra instance mới (GET / → 200)
4. Instance mới pass health check → đưa vào Target Group
5. Instance cũ bị terminate
6. Lặp lại cho đến khi tất cả instance được thay mới

4.5 Minh Chứng

❖ CD Pipeline chạy thành công

The screenshot shows the GitHub Actions interface for a pipeline named 'CD Pipeline #5'. The pipeline was triggered via workflow run 13 hours ago and is in a 'Success' status. The total duration is 6m 37s. The main action shown is 'Deploy to ASG' which completed successfully. Below the action, there are annotations including a warning about deprecated Node.js 20 actions.

❖ ASG Instance refresh history trên AWS

The screenshot shows the AWS Management Console 'Auto Scaling group details' page for the 'goldenowl-devops-asg'. The 'Instance refresh history' section displays a table of refresh events, all of which were successful.

Instance refresh ID	Instance refresh status	Rollback status	Status reason	Percentage completed
38ee448d-a12d-4407-b08d-ae0f9625f00a	Successful	-	-	100%
3cc43586-f1c7-4d81-8b45-f8f2e52c05d2	Successful	-	-	100%
b41a9071-05a0-46a0-bc28-afe9391757e9	Successful	-	-	100%
f90f2141-246a-43ef-8c3f-16028c69a003	Successful	-	-	100%
614b9a9e-93ae-49b9-a2d9-de3c21b13cfc	Successful	-	-	100%
4c9d1030-1ada-48c2-9c39-7064fe5bd6fb	Successful	-	-	100%

5. Infrastructure as Code (Terraform)

5.1 Tổng quan

Toàn bộ hạ tầng AWS được quản lý bằng Terraform (terraform/):

Resource	Type	Mục đích
aws_lb	Application Load Balancer	Phân phối traffic (port 80)
aws_lb_target_group	Target Group	Health check, route đến EC2 (port 3000)
aws_lb_listener	Listener	Forward HTTP:80 → Target Group
aws_launch_template	Launch Template	Template cấu hình EC2 + userdata
aws_autoscaling_group	ASG	Quản lý 1-3 instances, rolling update
aws_autoscaling_policy	Scaling Policy	Scale theo CPU (target 70%)
aws_security_group (x2)	Security Groups	ALB SG + App SG
aws_key_pair	Key Pair	SSH key tự generate (debugging)

5.2 Cấu trúc file

```
terraform/  
├── main.tf           # Toàn bộ resource definitions  
├── variables.tf      # Input variables (region, instance_type, ASG config)  
├── outputs.tf        # Output values (ALB DNS, ASG name, SSH key)  
├── userdata.sh       # Script bootstrap EC2 (install Docker + run app)  
└── terraform.tfvars.example # Ví dụ cấu hình
```

5.3 Userdata script

EC2 instance khi khởi động sẽ tự động:

```
#!/bin/bash  
# 1. Cài đặt Docker  
apt-get update -y && apt-get install -y docker-ce ...  
  
# 2. Pull Docker image mới nhất  
docker pull ${dockerhub_image}:latest  
  
# 3. Chạy container  
docker run -d --name app --restart unless-stopped \  
-p 3000:3000 ${dockerhub_image}:latest
```

5.4 Security Groups

- ALB Security Group:
 - Inbound: HTTP (80), HTTPS (443) ← 0.0.0.0/0 (public)
 - Outbound: All ← 0.0.0.0/0
- App Security Group:
 - Inbound: SSH (22) ← 0.0.0.0/0 (debugging)
 - App (3000) ← ALB SG only (restricted)
 - Outbound: All ← 0.0.0.0/0

Lưu ý : Port 3000 trên EC2 chỉ cho phép traffic từ ALB Security Group, không mở public. Users chỉ truy cập qua ALB (port 80).

6. Load Balancer & Auto Scaling

6.1 Application Load Balancer (ALB)

Chức năng:

- Nhận traffic từ Internet qua port 80
- Phân phối request đều đến các EC2 instances
- Health check mỗi 30s đến GET / trên port 3000
- Tự động loại bỏ instance unhealthy khỏi rotation

Health check Configuration:

Path: /
Port: 3000
Protocol: HTTP
Healthy threshold: 2 lần liên tiếp pass
Unhealthy threshold: 3 lần liên tiếp fail
Timeout: 5 giây
Interval: 30 giây
Success code: 200

6.2 Auto Scaling Group (ASG)

Configuration:

Min Size: 1 instance
Max Size: 3 instances
Desired Capacity: 2 instances
Health Check Type: ELB (ALB-based)
Grace Period: 120 giây

Scaling Policy:

- **Loại:** Target Tracking Scaling
- **Metric:** ASGAverageCPUUtilization
- **Target:** 70%
- Khi CPU trung bình > 70% → ASG tự động scale out thêm instance
- Khi CPU trung bình < 70% → ASG tự động scale in bớt instance

6.3 Instance refresh (Rolling Update)

Strategy: Rolling
Min Healthy Percentage: 50%
Instance Warmup: 120 giây

- Đảm bảo luôn có $\geq 50\%$ instances healthy trong quá trình update
- Chờ 120s cho instance mới hoàn thành khởi động trước khi kiểm tra health

6.4 Minh Chứng

❖ AWS Application Load Balancer

The screenshot displays the AWS Management Console interface for an Application Load Balancer. The left sidebar shows the navigation menu with categories like Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is titled 'goldenowl-devops-alb' and includes a 'Details' section with the following information:

- Load balancer type:** Application
- Status:** Active
- Scheme:** Internet-facing
- Hosted zone:** Z1LMS91P8CMLE5
- VPC:** vpc-0c3034d386c6ba035
- Availability Zones:** subnet-0c77160db4321df79 (ap-southeast-1a), subnet-0e859b2c48a4d88676 (ap-southeast-1c), subnet-06630f1ce62f8125b (ap-southeast-1b)
- Load balancer IP address type:** IPv4
- Date created:** May 29, 2026, 16:06 (UTC+07:00)
- Load balancer ARN:** arn:aws:elasticloadbalancing:ap-southeast-1:798836978890:loadbalancer/app/goldenowl-devops-alb/561f8f8cecf87546
- DNS name:** goldenowl-devops-alb-903237556.ap-southeast-1.elb.amazonaws.com (A Record)

Below the details, there are tabs for 'Listeners and rules', 'Network mapping', 'Resource map', 'Security', 'Monitoring', 'Integrations', 'Attributes', 'Capacity', and 'Tags'. The 'Listeners and rules' tab is active, showing a single listener with a status of 'Active' and a 'Manage rules' button. At the bottom, there are buttons for 'Manage rules', 'Manage listener', and 'Add listener'.

❖ AWS Autoscaling Group

The screenshot displays the AWS Management Console for the 'goldenowl-devops-asg' Autoscaling Group. The left sidebar shows the navigation menu with categories like EC2, Instances, Images, Elastic Block Store, and Network & Security. The main content area is titled 'goldenowl-devops-asg' and includes a 'Capacity overview' section with details such as 'Desired capacity: 1', 'Scaling limits: 1 - 3', and 'Status: At desired capacity'. Below this, there's a 'Launch template' section showing details like 'AMI ID: ami-0a010a0312b1400ee', 'Instance type: t2.micro', and 'Security group IDs: sg-03ef9f77cd5370954'. The bottom of the console shows a footer with copyright information and links to Privacy, Terms, and Cookie preferences.

7. Ứng dụng:

URL: <http://goldenowl-devops-alb-903237556.ap-southeast-1.elb.amazonaws.com>

The screenshot shows a web browser window with the address bar displaying the URL 'goldenowl-devops-alb-903237556.ap-southeast-1.elb.amazonaws.com'. The browser's security status is indicated as 'Không bảo mật' (Not secure). Below the address bar, there's a button labeled 'Tạo bản in đẹp' (Create beautiful print) with a checkmark icon.

```
{
  "message": "Welcome warriors to Golden Owl!"
}
```

8. Kết luận

#	Thành phần	Mô tả
1	Containerization	Multi-stage Dockerfile, Alpine-based, non-root, health check
2	CI pipeline	Automated test, lint, build, push với caching optimization
3	CD pipeline	Zero-downtime rolling deployment qua ASG Instance Refresh
4	Infrastructure	Toàn bộ hạ tầng managed bằng Terraform
5	Load Balancer	AWS ALB phân phối traffic với health checks
6	Auto Scaling	CPU-based scaling (1-3 instances)
7	Security	Security Groups
8	Documentation	Flow diagram (draw.io), README, báo cáo chi tiết